

Paterni Ponašanja - Seminarski Rad / Analiza

1.1. Analiza trenutnog stanja i potreba za uvođenjem paternna ponašanja

Trenutna implementacija sistema "InterTrips" unutar klase RezervacijaController oslanja se na proceduralni pristup gdje jedna metoda (akcija) obavlja previše različitih zadataka. Unutar metode za kreiranje rezervacije nalazi se logika za validaciju podataka, računanje konačne cijene sa popustima, te provjera i izvršavanje načina plaćanja. Ovakav dizajn direktno krši princip jedinstvene odgovornosti (SRP), jer kontroler upravlja poslovnom logikom koja ne bi trebala biti u njegovom domenu.

Također, krši se i Open/Closed princip (OCP), s obzirom na to da bi uvođenje bilo kakve izmjene, poput novog načina plaćanja (npr. PayPal) ili drugačijeg obračuna popusta, zahtijevalo direktno mijenjanje i prepravljavanje postojećeg koda unutar kontrolera. Kako bi se postigao labaviji spoj (loose coupling) između komponenti i omogućila lakša skalabilnost aplikacije, u sistem je potrebno uvesti arhitektonske paterne ponašanja koji će izmjestiti i enkapsulirati ovu logiku.

1.2. Prijedlog i opravdanje implementacije specifičnih paternna ponašanja

1) Strategy Pattern (Strategija)

Primjenjuje se u podsistemu za plaćanje i kalkulaciju cijena kako bi se uklonili komplikovani uslovi (if-else grananja) za različite metode uplate. Kreira se zajednički interfejs sa metodom za procesiranje uplate, dok posebne klase enkapsuliraju logiku za kartično plaćanje i plaćanje na rate. Na ovaj način, dodavanje bilo kojeg novog načina plaćanja u budućnosti neće zahtijevati nikakvu izmjenu postojećeg koda u kontroleru.

2) Observer Pattern (Posmatrač)

Uvodi se unutar podsistema za upravljanje kapacitetima i automatsko slanje notifikacija nakon uspješne rezervacije. Umjesto da kontroler ručno poziva različite servise, servis rezervacije preuzima ulogu subjekta koji nakon uplate samo emituje događaj da je rezervacija potvrđena. Klase zadužene za hotele, letove i slanje mailova postaju posmatrači koji potpuno nezavisno i asinhrono reaguju na taj događaj.

3) State Pattern (Stanje)

Koristi se za upravljanje životnim ciklusom rezervacije kroz njena stanja (Kreirana, Potvrđena, Otkazana), gdje akcije poput otkazivanja zavise od trenutnog statusa. Svako stanje se izdvaja u zasebnu klasu koja sama implementira dozvoljenu logiku i upravlja prelazima u naredna validna stanja. Time se izbjegavaju teške uslovne petlje unutar domenskih modela.

4) Template Method Pattern (Šablonska metoda)

Primjenjuje se u podsistemu za registraciju različitih tipova korisničkih naloga (Klijent, Agent, Administrator) koji dijele bazične korake poput unosa podataka i hashinga šifre. Apstraktna bazna klasa fiksira strukturu i redoslijed algoritma registracije. Specifični koraci se prepuštaju podklasama, čime se sprječava dupliranje koda i garantuje uniforman sigurnosni protokol.

5) Visitor Pattern (Posjetilac)

Omogućava definisanje novih operacija nad strukturama objekata bez mijenjanja samih klasa nad kojima operiše. Implementiran je kroz servis koji obilazi kompleksne podatke o rezervaciji (putnici, smještaj, cijene) i iz njih dinamički generiše PDF potvrdu sa QR kodom. Originalne domenske klase tako ostaju potpuno netaknute i odvojene od logike za crtanje dokumenata.

6) Command Pattern (Komanda)

Pretvara zahtjev u samostalni objekat, što omogućava stavljanje u red čekanja i odloženo izvršavanje operacija. U podsistemu za slanje notifikacija, zahtjev za slanje e-maila se pretvara u komandu i smješta u izolovani red čekanja (queue). Aplikacija odmah vraća odgovor korisniku bez usporavanja interfejsa, dok pozadinski proces naknadno preuzima i izvršava komandu.

7) Chain of Responsibility Pattern (Lanac odgovornosti)

Organizovan je kao lista objekata gdje svaki objekat odlučuje da li će obraditi HTTP zahtjev ili ga proslijediti narednom u nizu. Patern je integrisan kroz ASP.NET Core Middleware komponente koje provjeravaju zahtjev u tačnom redoslijedu: rutiranje, autentifikacija, pa autorizacija. Ukoliko korisnik nema odgovarajuću ulogu, lanac se automatski prekida i zahtjev se odbija prije nego što stigne do baze podataka.

8) Mediator Pattern (Posrednik)

Enkapsulira komunikaciju između objekata tako da oni ne moraju poznavati internu strukturu jedni drugih, čime se postiže labavo vezivanje. Uloga posrednika ostvarena je kroz samu MVC arhitekturu gdje kontroler djeluje kao centralna tačka između modela i pogleda (View). Komponente modela i pogleda nemaju nikakvu direktnu komunikaciju niti se međusobno

poznaju.

9) Iterator Pattern (Iterator)

Omogućava sekvencijalan pristup elementima neke kolekcije bez poznavanja njenog unutrašnjeg sastava i organizacije u memoriji. Ugrađen je kroz .NET interfejs i petlju foreach prilikom prolaska kroz kolekciju putnika. Servis za generisanje dokumenata može uniformno čitati podatke o putnicima bez obzira na to da li su oni u pozadini sačuvani kao lista ili niz.

10) Interpreter Pattern (Prevodilac)

Služi za interpretaciju izraza napisanih za određenu upotrebu definisanjem njihove gramatike. Realizovan je kroz LINQ stabla izraza (Expression Trees) unutar Entity Framework Core ORM-a. Kada se u aplikaciji vrši pretraga ili filtriranje podataka pomoću lambda izraza, framework taj kod interpretira i automatski prevodi u izvorni SQL upit prilagođen bazi podataka.

11) Memento Pattern (Podsjetnik)

Omogućava spašavanje i restauraciju prethodnog unutrašnjeg stanja objekta bez narušavanja enkapsulacije. Primjenjuje se kroz mehanizam očuvanja stanja modela u HTTP zahtjevima prilikom validacije web formi. Ako korisnik unese pogrešne ili nepotpune podatke, kontroler uzima snimak trenutnog stanja unesenog modela i vraća ga nazad na formu kako se tekstualna polja ne bi ispraznila.